

CHƯƠNG 04: TÌM KIẾM TRONG MÔI TRƯỜNG PHỨC TẠP

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 4

- ◇ 4.1 Tìm kiếm Cục bộ và Bài toán tối ưu hóa (Local Search)
- ◇ 4.2 Tìm kiếm Cục bộ trong Không gian Liên tục
- ◇ 4.3 Tác nhân với Hành động Không tất định (Nondeterministic Actions)
- ◇ 4.4 Tìm kiếm với Quan sát Một phần (Partially Observable)
- ◇ 4.5 Môi trường chưa biết & Khám phá Trực tuyến (Online Search)

4.1 Tìm kiếm Cục bộ (Local Search)

◇ Ở các chương trước, ta quan tâm đến **Đường đi (Path)** tới đích. Tuy nhiên trong nhiều bài toán, đường đi không quan trọng, chỉ có **Trạng thái đích (Goal state)** mới là thứ cần tìm (Ví dụ: 8-Queens, Lập lịch, Thiết kế mạch).

◇ **Tìm kiếm cục bộ (Local search)**: Chỉ hoạt động với một (hoặc một vài) trạng thái hiện tại, và di chuyển sang các trạng thái lân cận thay vì lưu trữ toàn bộ cây tìm kiếm.

◇ **Ưu điểm**:

- Tốn cực kì ít bộ nhớ (thường là hằng số $\mathcal{O}(1)$).
- Có thể tìm ra nghiệm hợp lý trong các không gian trạng thái vô tận hoặc liên tục mà thuật toán truyền thống bó tay.

Cảnh quan Không gian trạng thái

◇ **Hàm mục tiêu (Objective function):** Đánh giá chất lượng của một trạng thái. Ta muốn tìm **Cực đại toàn cục (Global maximum)**.

◇ **Hàm chi phí (Cost function):** Ta muốn tìm **Cực tiểu toàn cục (Global minimum)**.

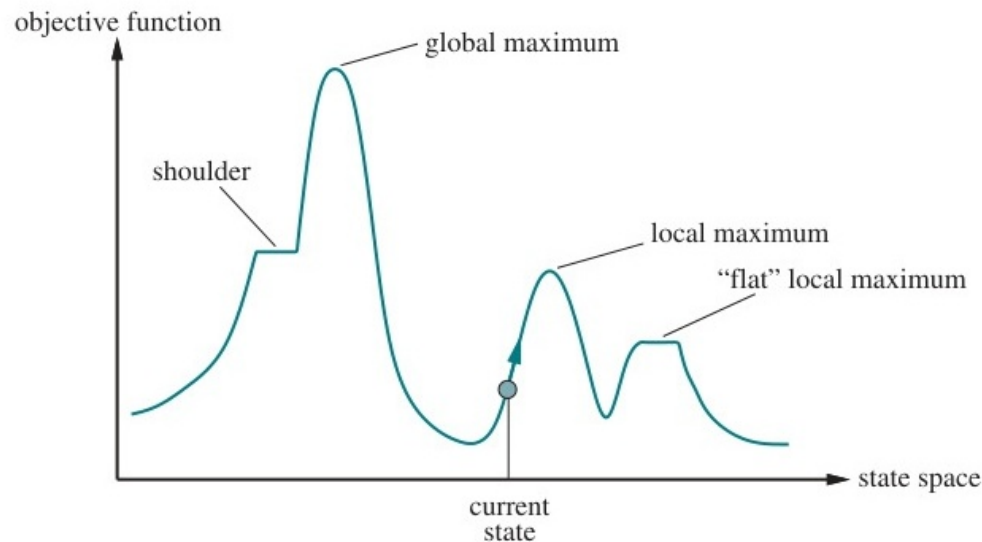


Figure 1: Mô phỏng cảnh quan không gian trạng thái 1 chiều (State-space landscape).

Thuật toán Leo đồi (Hill-climbing Search)

◇ **Chiến lược:** Liên tục di chuyển theo hướng dốc lên (tăng giá trị của hàm mục tiêu). Dừng lại khi lên đến đỉnh (không có hàng xóm nào cao hơn).

◇ Còn được gọi là "Greedy local search" vì nó chỉ lấy trạng thái hàng xóm tốt nhất hiện tại.

◇ **Khuyết điểm - Dễ bị kẹt tại:**

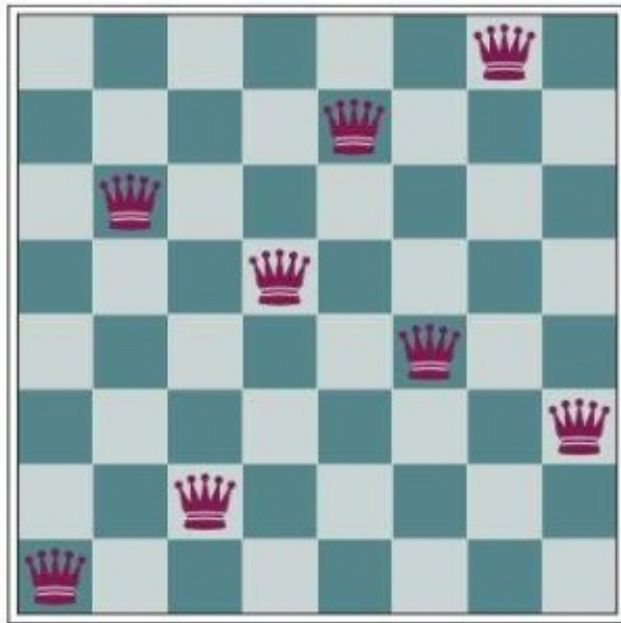
- **Cực đại cục bộ (Local maxima):** Đỉnh nhỏ hơn đỉnh cao nhất thực sự.
- **Sườn dốc bằng (Ridges):** Đỉnh nhọn nhưng hẹp kéo dài.
- **Cao nguyên (Plateaux):** Một vùng phẳng khiến thuật toán đi lang thang do không xác định được hướng dốc.

Mã giả Thuật toán Leo đồi

```
function HILL-CLIMBING(problem) returns a state that is a local maximum  
  current  $\leftarrow$  problem.INITIAL  
  while true do  
    neighbor  $\leftarrow$  a highest-valued successor state of current  
    if VALUE(neighbor)  $\leq$  VALUE(current) then return current  
    current  $\leftarrow$  neighbor
```

Figure 2: Thuật toán tìm kiếm leo đồi (Hill-climbing search).

Ví dụ Thuật toán Leo đồi: 8-Queens



(a)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	13	16	13	16	16
14	17	15	14	16	16	16	16
17	16	18	15	15	14	16	16
18	14	15	15	14	16	16	16
14	14	13	17	12	14	12	18

(b)

Figure 3: Sử dụng Heuristic đếm số lượng cặp hậu tấn công nhau (Min-conflicts).

Khó khăn của Thuật toán Leo đồi

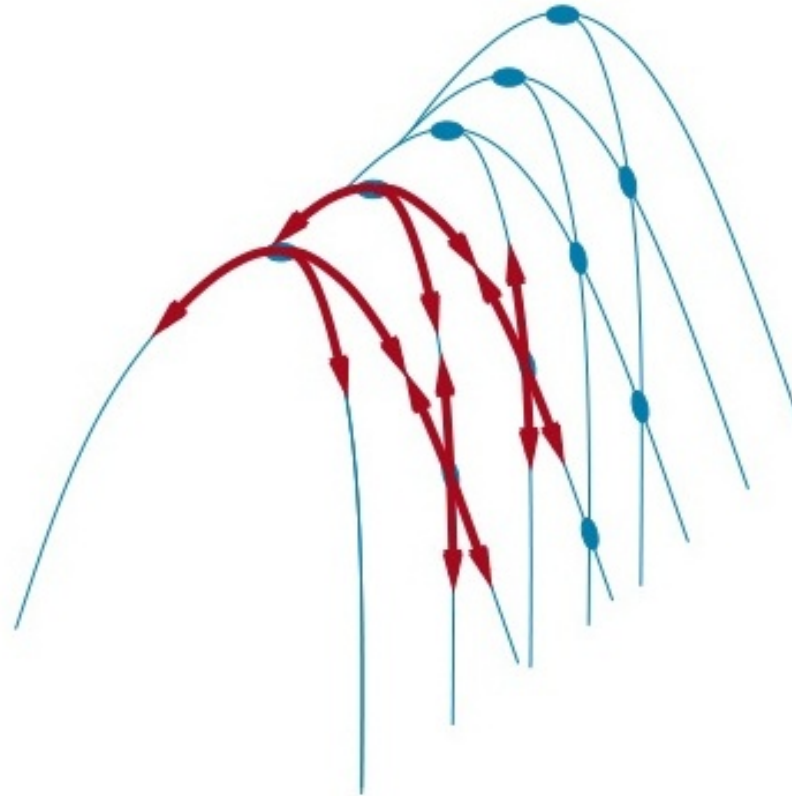


Figure 4: Minh họa sườn núi (Ridges) gây khó khăn cho việc leo đồi.

Luyện kim nhân tạo (Simulated Annealing)

◇ Để thoát khỏi cực đại cục bộ, thỉnh thoảng thuật toán cần được phép đi "xuống dốc". Tuy nhiên, nếu cứ đi ngẫu nhiên thì sẽ không thể hội tụ.

◇ **Luyện kim nhân tạo:** Lấy cảm hứng từ quá trình tôi luyện kim loại (nung nóng rồi làm nguội từ từ).

- Nhiệt độ T sẽ giảm dần theo thời gian.
- Nếu hàng xóm n' tốt hơn n , luôn chấp nhận.
- Nếu hàng xóm n' tệ hơn (độ dốc $\Delta E < 0$), thuật toán vẫn chấp nhận nó với xác suất $p = e^{\Delta E/T}$.

◇ **Tính chất:** Khi T giảm từ từ, thuật toán **đảm bảo hội tụ** về cực đại toàn cục với xác suất xấp xỉ 1.

Thuật toán Luyện kim nhân tạo

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state  
  current  $\leftarrow$  problem.INITIAL  
  for  $t = 1$  to  $\infty$  do  
     $T \leftarrow$  schedule( $t$ )  
    if  $T = 0$  then return current  
    next  $\leftarrow$  a randomly selected successor of current  
     $\Delta E \leftarrow$  VALUE(current) – VALUE(next)  
    if  $\Delta E > 0$  then current  $\leftarrow$  next  
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$ 
```

Figure 5: Thuật toán mô phỏng luyện kim (Simulated annealing).

Chùm tia Cục bộ và Thuật toán Di truyền

◇ **Local Beam Search (Tìm kiếm Chùm tia):** Thay vì 1 trạng thái, giữ lại k trạng thái. Ở mỗi bước, sinh tất cả các con của k trạng thái, và chỉ chọn ra k con tốt nhất để đi tiếp.

◇ **Genetic Algorithms (Thuật toán Di truyền - GA):** Là biến thể ngẫu nhiên, mô phỏng quá trình tiến hóa sinh học:

- **Quần thể (Population):** Tập hợp k cá thể.
- **Lai ghép (Crossover):** Chọn 2 cá thể cha mẹ dựa trên độ thích nghi (Fitness function) để tạo ra thế hệ con cái (kết hợp các chuỗi gen).
- **Đột biến (Mutation):** Thay đổi ngẫu nhiên một phần nhỏ cấu trúc gen của cá thể con để duy trì tính đa dạng.

Mô phỏng Thuật toán Di truyền

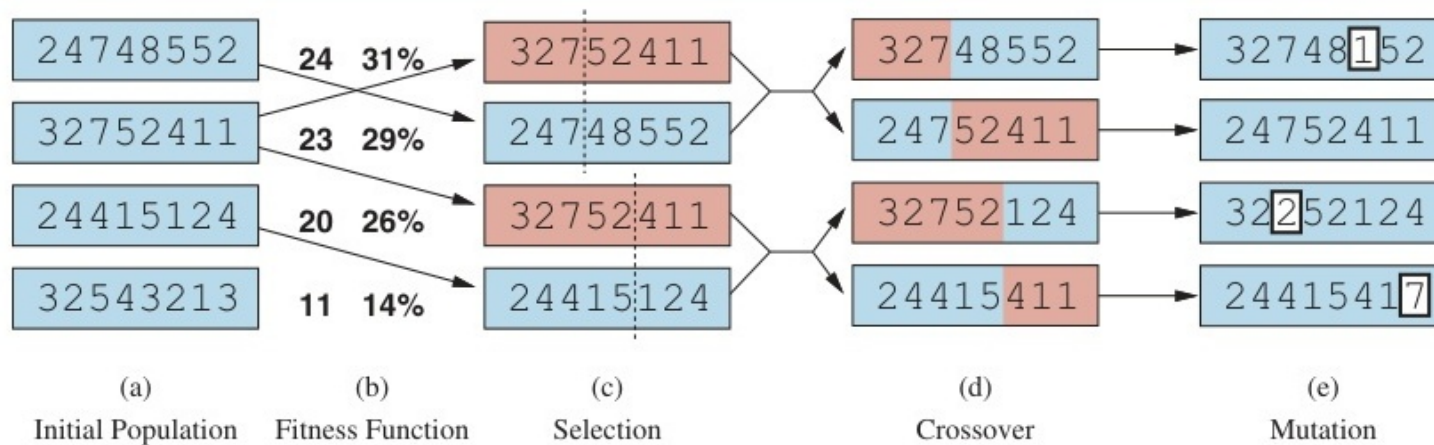


Figure 6: Tiến hóa (Đánh giá, Lựa chọn, Lai ghép, Đột biến) trong bài toán 8-Queens.

Lai ghép trong Thuật toán Di truyền

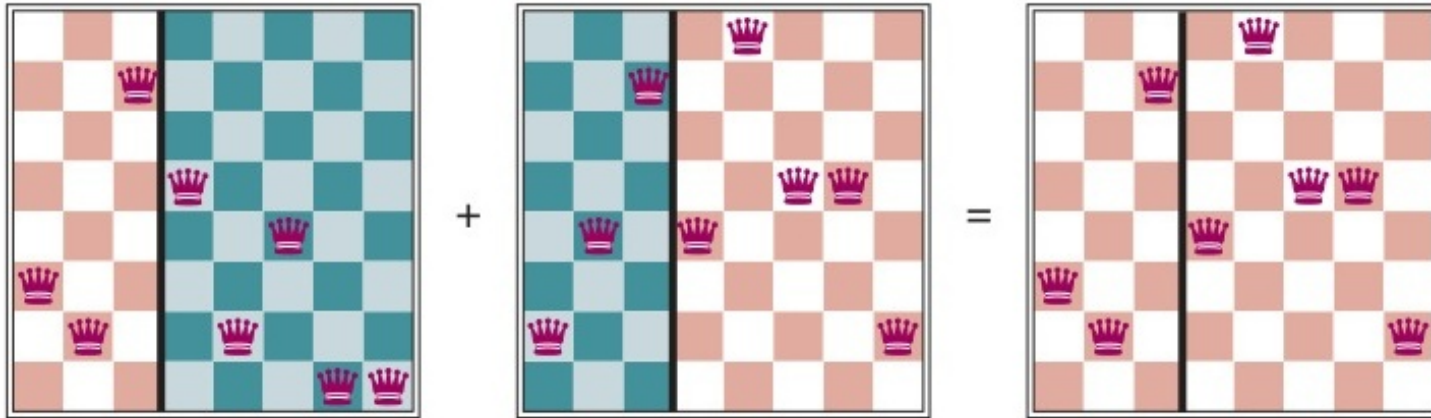


Figure 7: Các trạng thái 8 quân hậu tương ứng với hai cha mẹ trong quá trình lai ghép.

Mã giả Thuật toán Di truyền

```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights  $\leftarrow$  WEIGHTED-BY(population, fitness)
    population2  $\leftarrow$  empty list
    for i = 1 to SIZE(population) do
      parent1, parent2  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child  $\leftarrow$  REPRODUCE(parent1, parent2)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to population2
    population  $\leftarrow$  population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n  $\leftarrow$  LENGTH(parent1)
  c  $\leftarrow$  random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

Figure 8: Một thuật toán di truyền tiêu chuẩn.

4.2 Tìm kiếm Cục bộ trong Không gian Liên tục

◇ Rất nhiều bài toán thực tế có không gian trạng thái liên tục (Continuous state spaces). Ví dụ: Kiểm soát cánh tay Robot, Điều phối nhà máy.

◇ **Phương pháp xuống dốc Gradient (Gradient Descent):**

$$x_{new} = x - \alpha \nabla f(x)$$

- $\nabla f(x)$: Đạo hàm (hoặc vector gradient) chỉ hướng dốc lên lớn nhất.
- α : Tốc độ học (Learning rate / step size).

◇ Có thể áp dụng phương pháp Newton-Raphson để tận dụng đạo hàm bậc hai (Hessian matrix) giúp xác định điểm cực trị nhanh hơn.

4.3 Tác nhân với Hành động Không tất định

- ◇ Trong môi trường không tất định (Nondeterministic), hành động của tác nhân có thể dẫn đến nhiều hệ quả khác nhau ngoài dự đoán.
- ◇ Kế hoạch (Plan) không còn là 1 chuỗi hành động tuyến tính, mà là một **Chiến lược dự phòng (Contingency plan)**: "Nếu kết quả là A thì làm X, nếu là B thì làm Y".
- ◇ **Cây AND-OR:**
 - **OR-nodes:** Lựa chọn hành động của tác nhân (Ta có quyền quyết định).
 - **AND-nodes:** Kết quả sinh ra từ môi trường (Ta phải lường trước tất cả các hệ quả).
- ◇ Mục tiêu: Tìm cây con giải quyết được mọi kịch bản của các nút AND.

Không gian trạng thái máy hút bụi không tất định

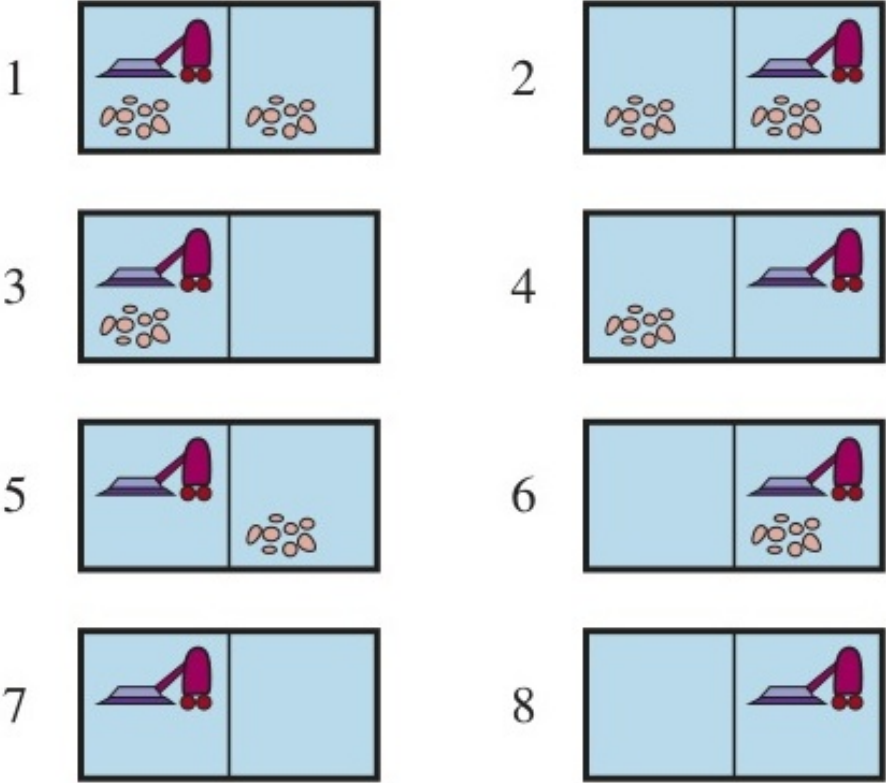


Figure 9: Tám trạng thái có thể có của thế giới chân không (trạng thái 7, 8 là đích).

Mô hình Cây AND-OR

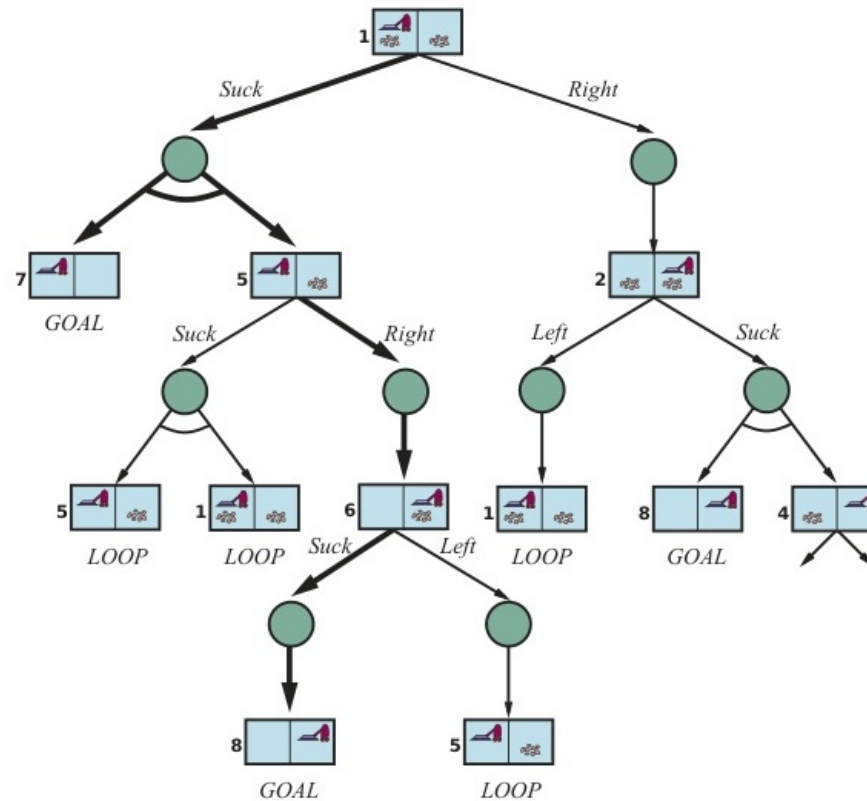


Figure 10: Mô hình cây AND-OR xử lý hệ quả không tắt định trong bài toán hút bụi.

Thuật toán Tìm kiếm Đồ thị AND-OR

```
function AND-OR-SEARCH(problem) returns a conditional plan, or failure  
  return OR-SEARCH(problem, problem.INITIAL, [])  
  
function OR-SEARCH(problem, state, path) returns a conditional plan, or failure  
  if problem.IS-GOAL(state) then return the empty plan  
  if IS-CYCLE(state, path) then return failure  
  for each action in problem.ACTIONS(state) do  
     $plan \leftarrow$  AND-SEARCH(problem, RESULTS(state, action), [state] + [path])  
    if  $plan \neq failure$  then return [action] + [plan]  
  return failure  
  
function AND-SEARCH(problem, states, path) returns a conditional plan, or failure  
  for each  $s_i$  in states do  
     $plan_i \leftarrow$  OR-SEARCH(problem,  $s_i$ , path)  
    if  $plan_i = failure$  then return failure  
  return [if  $s_1$  then  $plan_1$  else if  $s_2$  then  $plan_2$  else ... if  $s_{n-1}$  then  $plan_{n-1}$  else  $plan_n$ ]
```

Figure 11: Thuật toán tìm kiếm các đồ thị AND-OR.

Đồ thị tìm kiếm chứa chu trình

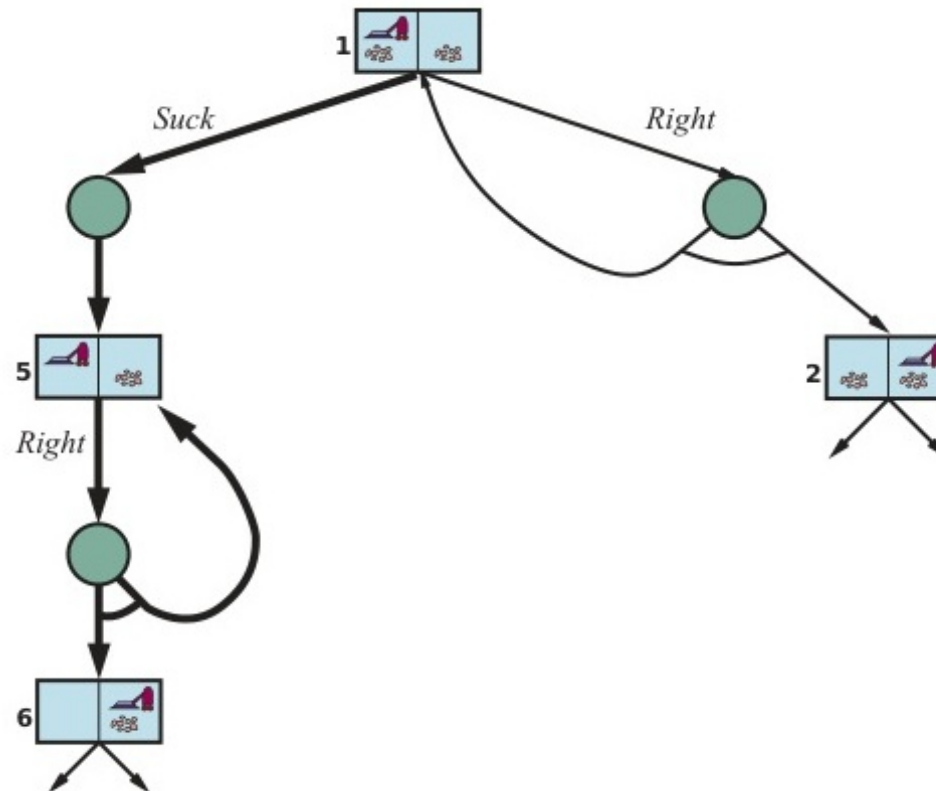


Figure 12: Một phần của đồ thị tìm kiếm cho một thế giới chân không trơn trượt có chứa chu trình.

4.4 Tìm kiếm với Quan sát Một phần

- ◇ Khi môi trường là Partially Observable (Quan sát một phần), tác nhân không biết chính xác mình đang ở trạng thái nào.
- ◇ Tác nhân duy trì khái niệm **Trạng thái Niềm tin (Belief state)**, đại diện cho tập hợp tất cả các trạng thái vật lý mà tác nhân có thể đang đứng ở thời điểm hiện tại.
- ◇ Quy trình:
 - b : Trạng thái niềm tin hiện tại.
 - Khi tác nhân thực hiện hành động a , cập nhật $b' = \text{Predict}(b, a)$.
 - Khi nhận được dữ liệu cảm biến (Percept) o , cập nhật tiếp $b'' = \text{Update}(b', o)$.
- ◇ Việc tìm kiếm chuyển đổi thành việc tìm đường trên "Không gian trạng thái niềm tin".

Dự đoán Trạng thái niềm tin (Không cảm biến)

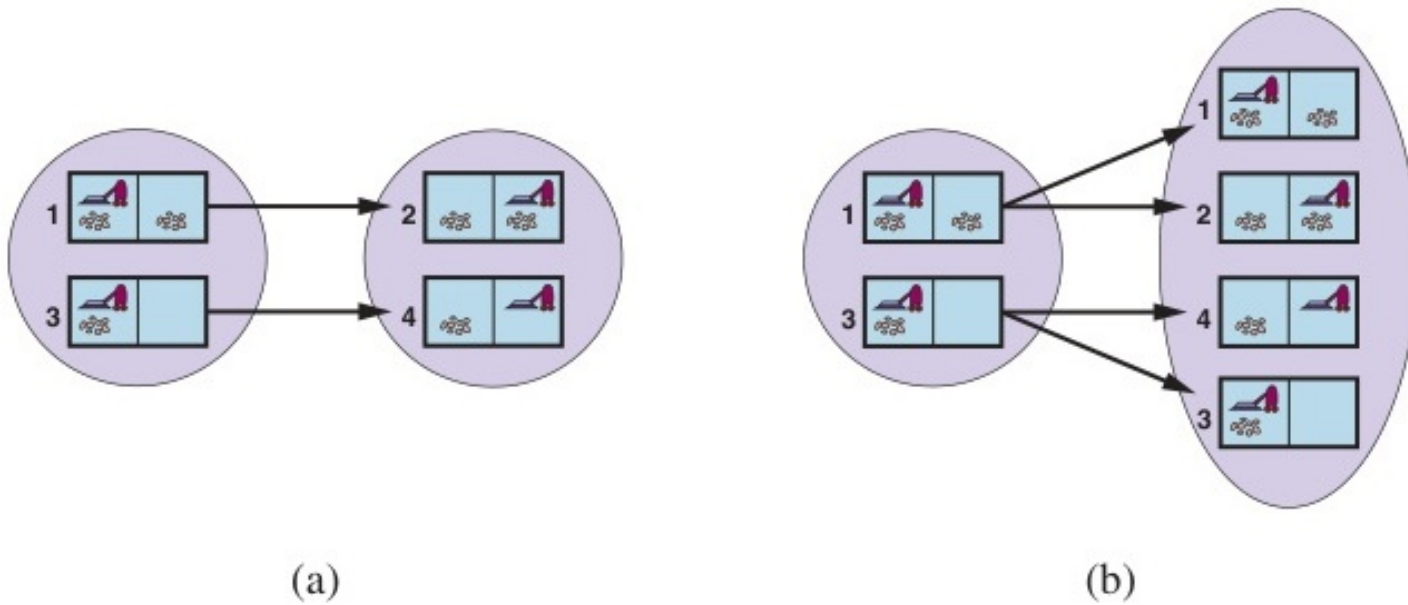


Figure 13: Dự đoán trạng thái niềm tin tiếp theo (Prediction).

Cập nhật Trạng thái Niềm tin

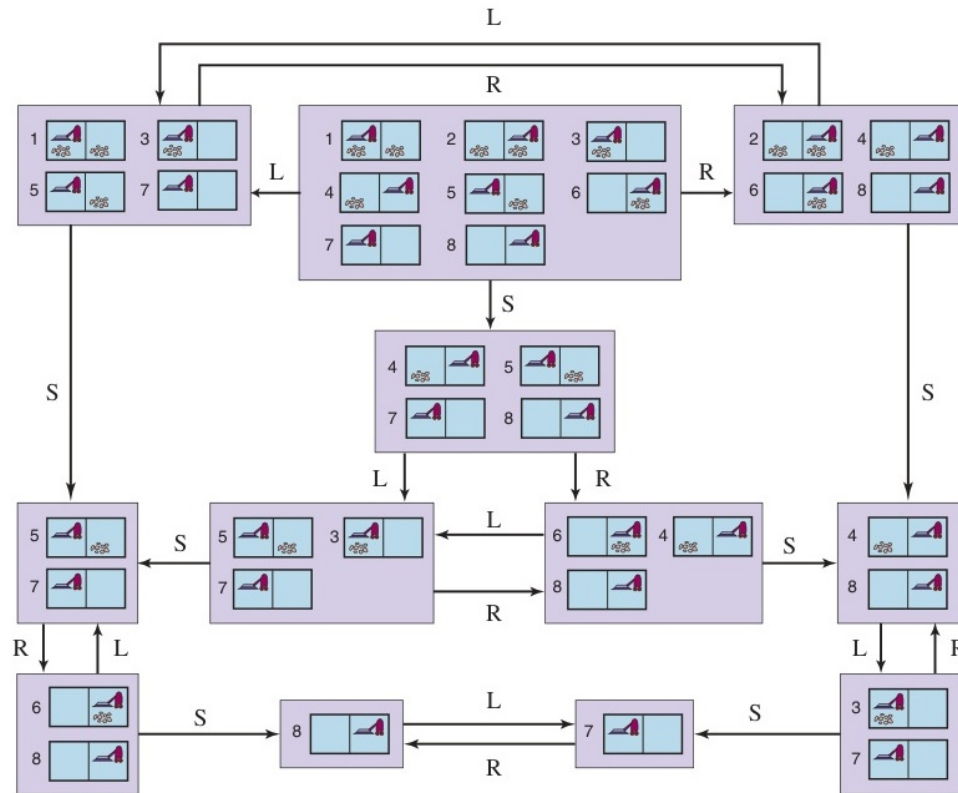


Figure 14: Sự chuyển đổi tập hợp Belief states của robot hút bụi khi không có cảm biến.

Quan sát một phần với Cảm biến cục bộ

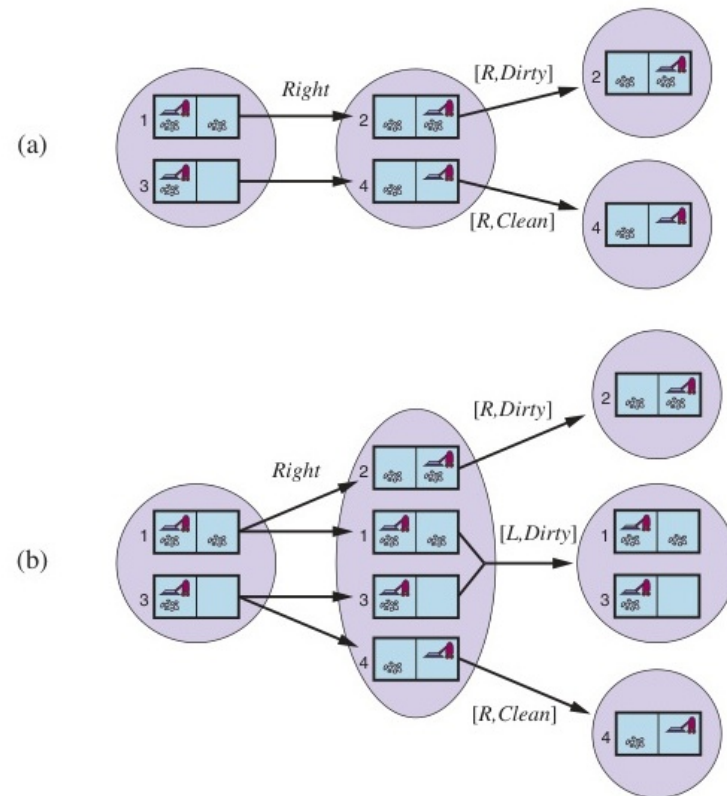


Figure 15: Sự chuyển đổi trạng thái niềm tin khi có cảm biến cục bộ.

Cây tìm kiếm AND-OR với Cảm biến cục bộ

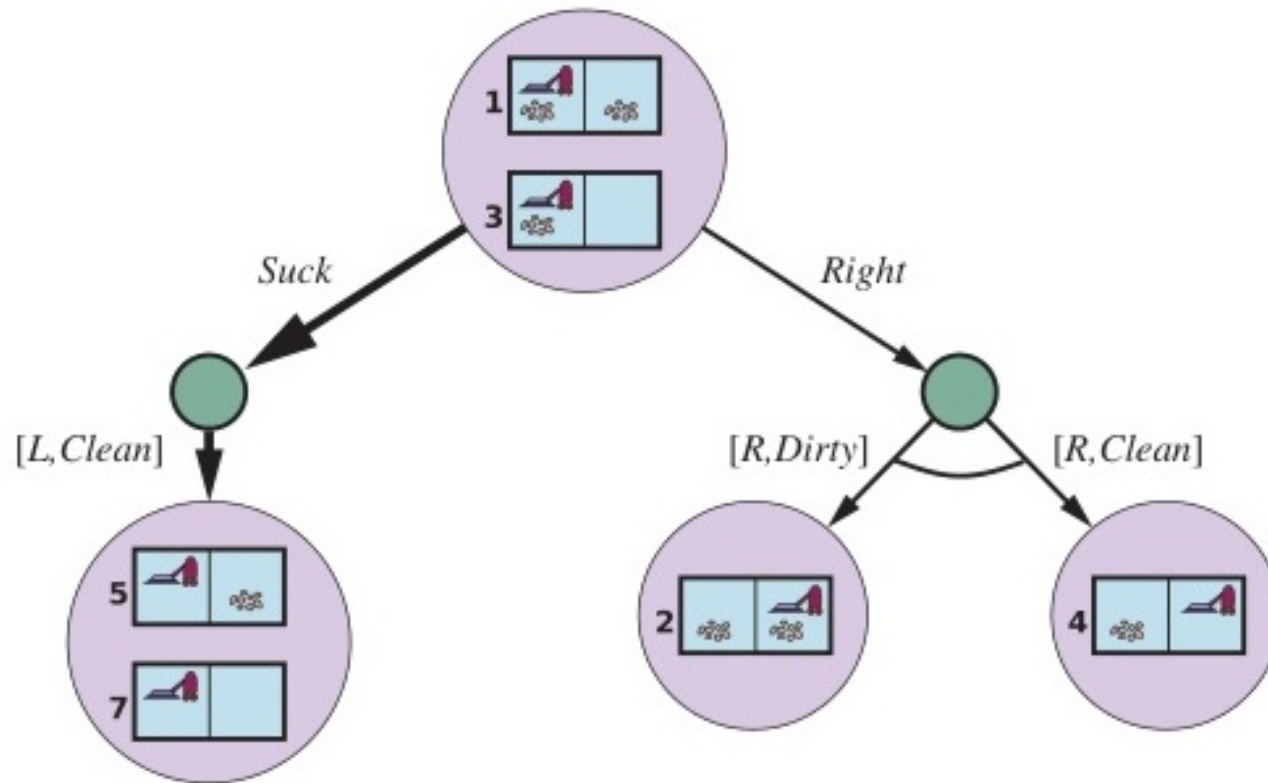


Figure 16: Cấp độ đầu tiên của cây tìm kiếm AND-OR khi trạng thái niềm tin là các nút.

Duy trì Trạng thái Niềm tin

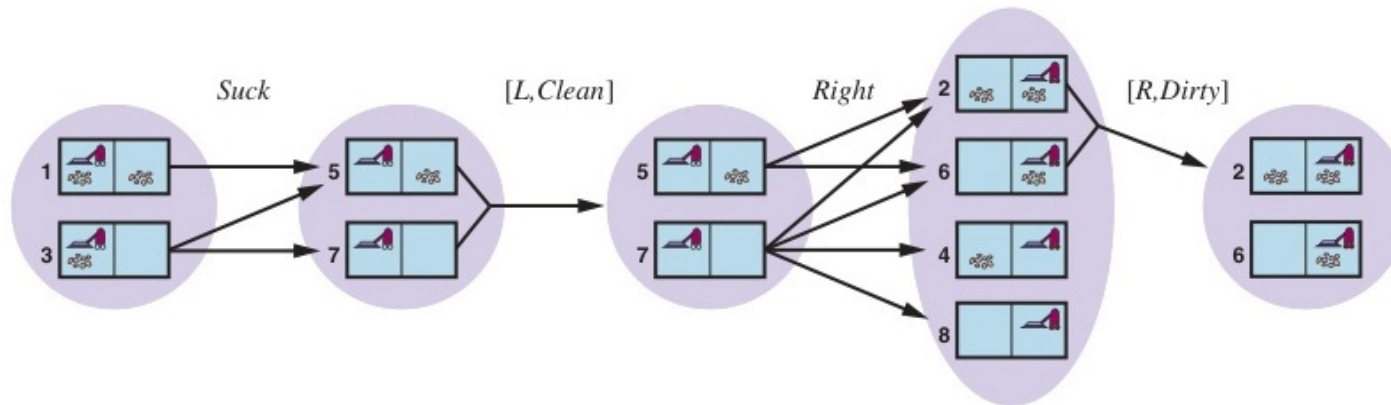
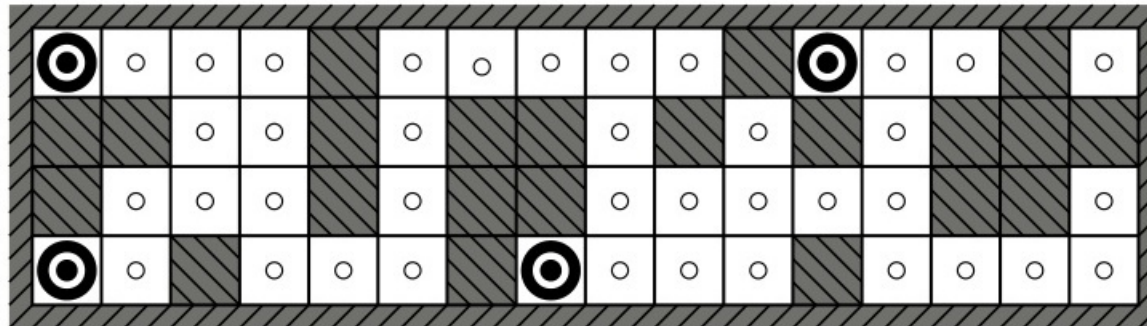
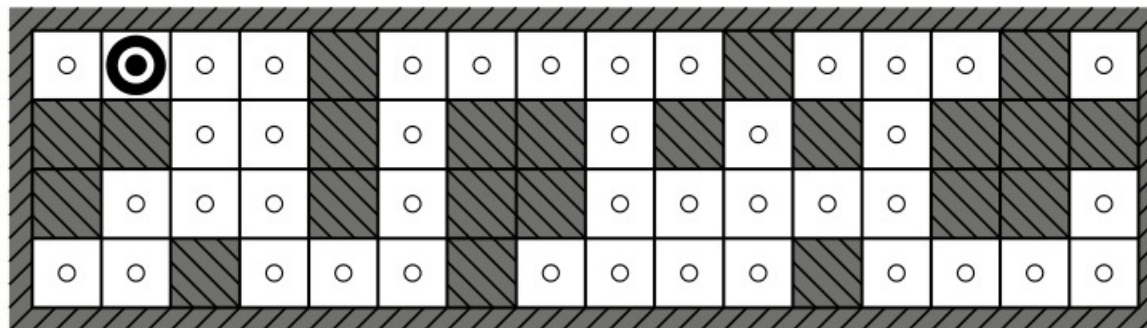


Figure 17: Chu kỳ dự đoán-cập nhật trong thế giới chân không.

Ứng dụng: Xác định vị trí Robot



(a) Possible locations of robot after $E_1 = 1011$



(b) Possible locations of robot after $E_1 = 1011, E_2 = 1010$

Figure 18: Các vị trí có thể có của robot (Localization).

4.5 Môi trường chưa biết & Khám phá Trực tuyến

◇ **Tìm kiếm ngoại tuyến (Offline search):** Tác nhân giải bài toán trong đầu trước, rồi nhắm mắt thực thi (Cần biết trước sơ đồ của môi trường).

◇ **Tìm kiếm trực tuyến (Online search):** Đan xen tính toán và thực thi (Interleave computation and action). Tác nhân phải dò dẫm trong môi trường hoàn toàn xa lạ (Unknown environment).

◇ Rủi ro lớn nhất là **Điểm mù vô tận (Dead ends)**: Tác nhân có thể đi vào ngõ cụt không thể thoát lui. ◇ Giải pháp: Sử dụng **Thuật toán Khám phá ngẫu nhiên (Random walk)** hoặc **Thuật toán Học LRTA*** (Learning Real-Time A*) để đánh giá và trừng phạt các nút dẫn tới ngõ cụt.

Hệ quả của Dead-Ends trong Online Search

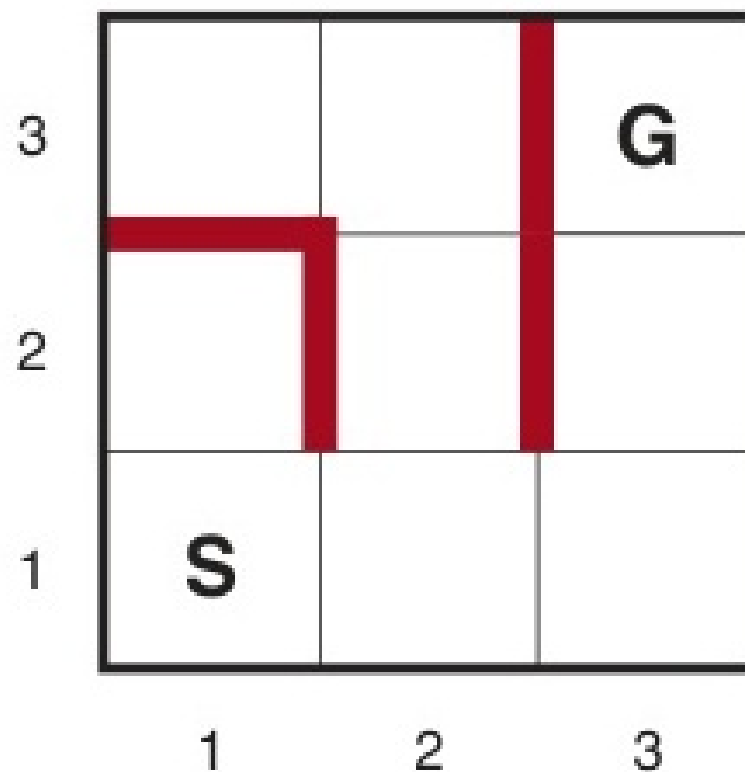


Figure 19: Robot bị kẹt (dead end) vĩnh viễn do không biết trước cấu trúc hành lang.

Điểm mù vô tận trong Môi trường thực tế

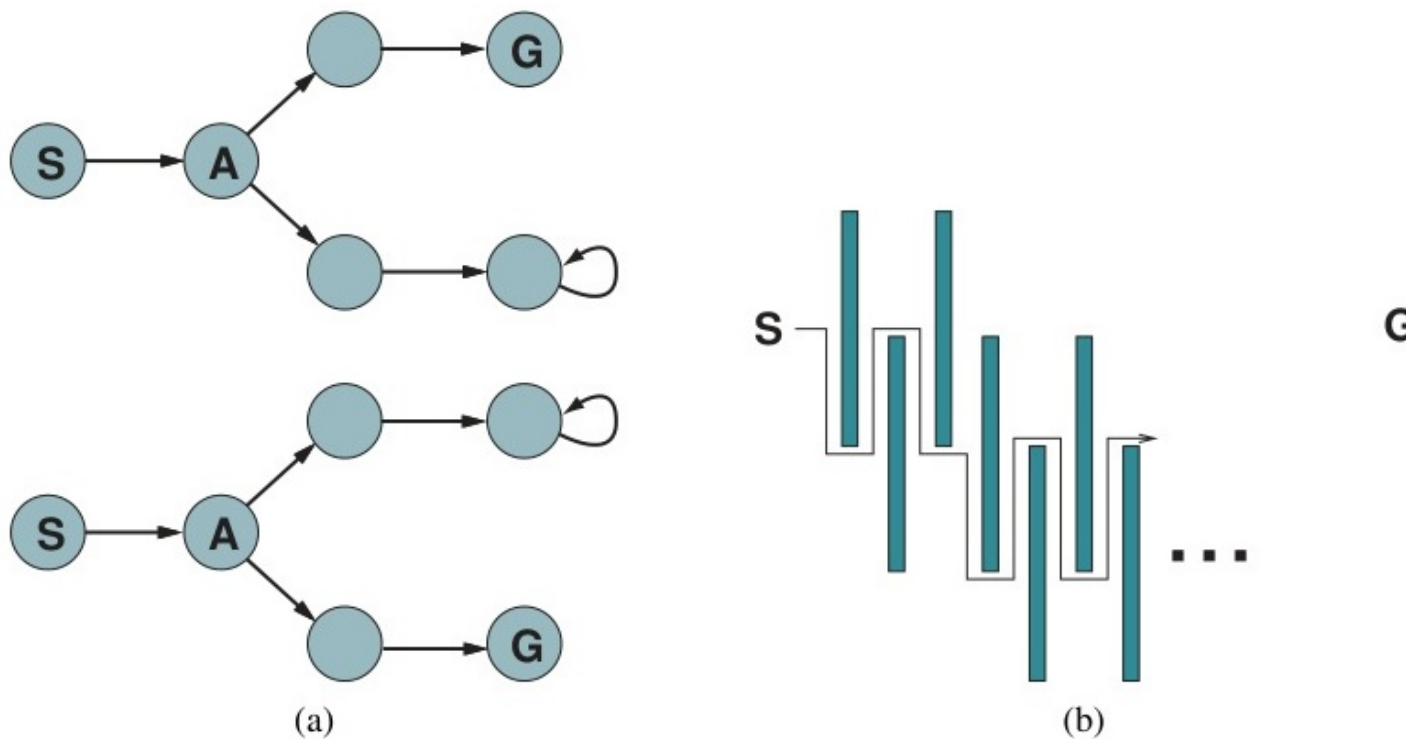


Figure 20: Hai không gian trạng thái có thể dẫn tác nhân vào ngõ cụt.

Khám phá theo Chiều sâu trực tuyến

function ONLINE-DFS-AGENT(*problem*, *s'*) **returns** an action
 s, *a*, the previous state and action, initially null
 result, a table mapping (*s*, *a*) to *s'*, initially empty
 untried, a table mapping *s* to a list of untried actions
 unbacktracked, a table mapping *s* to a list of states never backtracked to

if *problem.IS-GOAL*(*s'*) **then return** *stop*
if *s'* is a new state (not in *untried*) **then** *untried*[*s'*] $\leftarrow$ *problem.ACTIONS*(*s'*)
if *s* is not null **then**
 result[*s*, *a*] $\leftarrow$ *s'*
 add *s* to the front of *unbacktracked*[*s'*]
if *untried*[*s'*] is empty **then**
 if *unbacktracked*[*s'*] is empty **then return** *stop*
 a $\leftarrow$ an action *b* such that *result*[*s'*, *b*] = POP(*unbacktracked*[*s'*])) *s'* $\leftarrow$ null
else *a* $\leftarrow$ POP(*untried*[*s'*])
s $\leftarrow$ *s'*
return *a*

Figure 21: Thuật toán tìm kiếm trực tuyến sử dụng khám phá theo chiều sâu (Online DFS).

Khó khăn của thuật toán Ngẫu nhiên

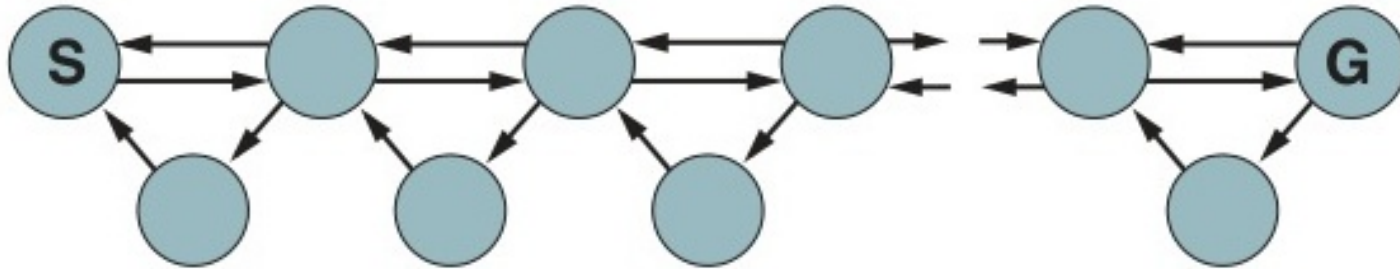


Figure 22: Môi trường khiến bước đi ngẫu nhiên mất số bước theo cấp số mũ.

Học heuristic trực tuyến (LRTA*)

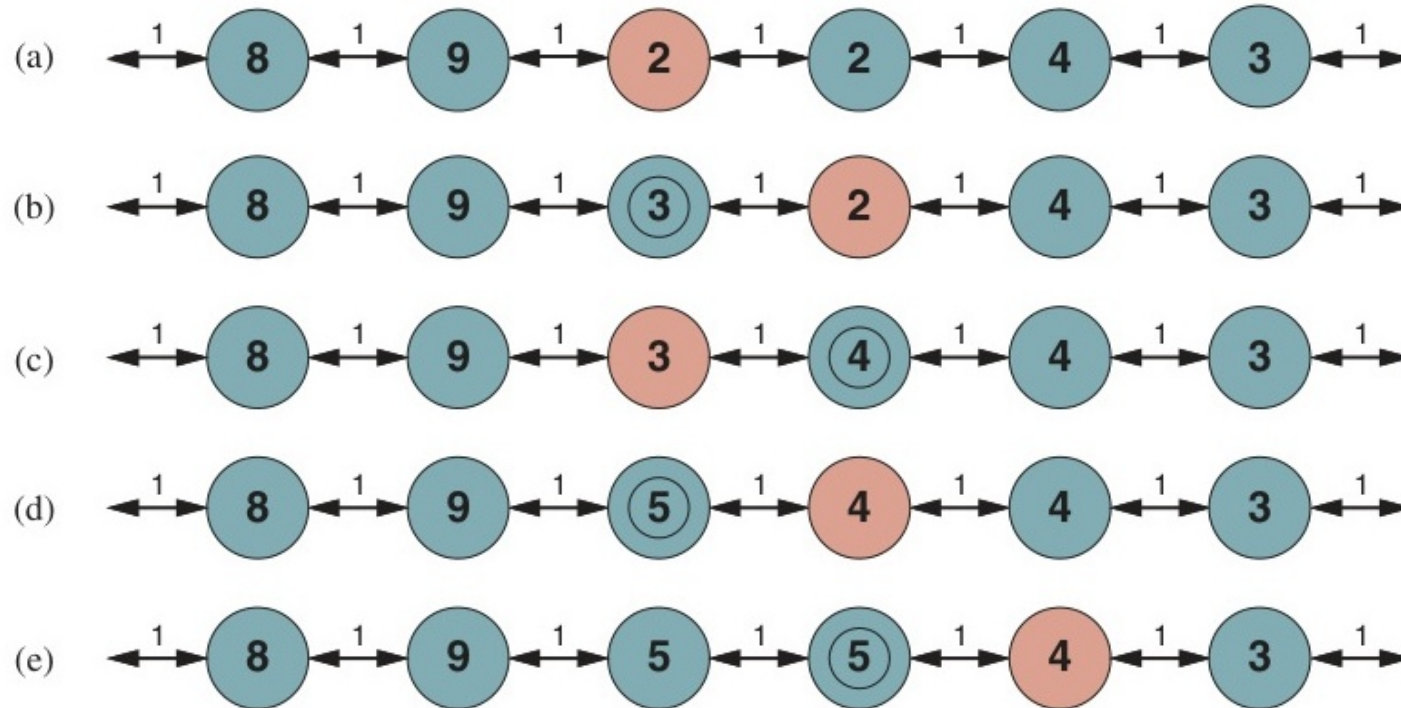


Figure 23: Năm lần lặp của Learning Real-Time A* (LRTA*).

Thuật toán LRTA*

```

function LRTA*-AGENT(problem,  $s'$ ,  $h$ ) returns an action
     $s$ ,  $a$ , the previous state and action, initially null
    result, a table mapping ( $s$ ,  $a$ ) to  $s'$ , initially empty
     $H$ , a table mapping  $s$  to a cost estimate, initially empty

    if IS-GOAL( $s'$ ) then return stop
    if  $s'$  is a new state (not in  $H$ ) then  $H[s'] \leftarrow h(s')$ 
    if  $s$  is not null then
         $result[s, a] \leftarrow s'$ 
         $H[s] \leftarrow \min_{b \in ACTIONS(s)} LRTA^*-COST(problem, s, b, result[s, b], H)$ 
     $a \leftarrow \operatorname{argmin}_{b \in ACTIONS(s)} LRTA^*-COST(problem, s', b, result[s', b], H)$ 
     $s \leftarrow s'$ 
    return  $a$ 

function LRTA*-COST(problem,  $s$ ,  $a$ ,  $s'$ ,  $H$ ) returns a cost estimate
    if  $s'$  is undefined then return  $h(s)$ 
    else return  $problem.ACTION-COST(s, a, s') + H[s']$ 

```

Figure 24: Mã giả thuật toán LRTA*-AGENT.

Tóm tắt Chương 4

- ◇ **Tìm kiếm Cục bộ (Local Search):** Thuật toán Leo đồi, Luyện kim nhân tạo (SA) và Di truyền (GA) giúp giải quyết các bài toán tối ưu trong không gian trạng thái cực kỳ khổng lồ với chi phí bộ nhớ chỉ $\mathcal{O}(1)$.
- ◇ **Không gian liên tục:** Sử dụng phương pháp Gradient Descent dựa trên vi phân $\nabla f(x)$.
- ◇ **Môi trường Không tất định:** Sinh ra các kế hoạch dự phòng (Contingency plans) thông qua cây tìm kiếm AND-OR.
- ◇ **Quan sát Một phần (Partial Observability):** Tìm kiếm trên không gian trạng thái niềm tin (Belief states).
- ◇ **Khám phá trực tuyến (Online Search):** Rất thiết thực trong môi trường chưa biết, kết hợp đan xen khám phá và hành động, nhưng phải cẩn trọng với các ngõ cụt (Dead ends).